

Resumen ejecutivo:

- NO existe cron job ni trigger de servidor que cancele pedidos de almuerzo por expiración horaria. Los crons de Vercel solo ejecutan auto-facturación y consulta SUNAT.
- Las anulaciones automáticas provienen del FRONTEND cuando falla el INSERT de la transacción (deuda) después de crear lunch_orders — patrón no-atómico de 2 pasos.
- En hora punta (7–9 AM), fn_sync_student_balance + contención de locks provocan timeouts; el cliente interpreta fallo y genera una AGENCIA_CANCELADA=true
- El panel de cocina muestra pedidos "Anulados" vía Realtime UPDATE sin remover filas canceladas de la lista. Lima Café 28 / Tuqui POS — parent-portal-connect
- La PWA tiene skipWaiting+clientsClaim, pero VersionChecker DELIBERADAMENTE no recarga; usuarios siguen ejecutando bundle JS viejo en memoria. Fecha del informe: 18 de junio de 2026
- Usuarios en builds anteriores a VersionChecker nunca ven el botón de actualización.

Metodología: análisis estático del repositorio real (SQL, triggers, RPC, Edge Functions, vercel.json, PWA/Workbox, componentes React). Sin suposiciones.



TASK 1 — Causa raíz de anulaciones/cancelaciones

1.1 Hallazgo principal: NO hay auto-cancelación server-side por tiempo

Se revisaron migraciones SQL, triggers, RPCs, Edge Functions y vercel.json. No hay job que pase pedidos a cancelled por TTL/expiración.

- vercel.json crons: /api/cron/auto-invoice (04:50 UTC) y /api/cron/check-invoice-status (c/5 min). Solo facturación.
- Triggers en lunch_orders: trg_validate_lunch_order_deadline (BLOQUEA INSERT, no cancela), trg_lunch_orders_prepayment (asigna payment_flow_state), trg_lunch_order_cancellation_cascade (REACCIONA a cancelación, no la inicia).
- void_pending_lunch_order_v2 y cancel_lunch_order*: solo se invocan por acción explícita de usuario/admin.
- cancelled_expired existe como valor de enum en payment_flow_state pero NO hay código que lo asigne automáticamente.

1.2 Lógica exacta que SÍ cambia pedidos a anulado (AUTO)

Patrón común: INSERT lunch_orders OK ! INSERT transactions FALLA ! UPDATE is_cancelled=true, status=cancelled.

A) src/components/lunch/OrderLunchMenus.tsx — líneas 565-572

```
if (transactionError) {
  await supabase.from("lunch_orders").update({
    is_cancelled: true, status: "cancelled",
    cancellation_reason: "AUTO: transacción de almuerzo fallida al crear pedido"
  }).eq("id", insertedOrder.id);
}
```

B) src/components/lunch/UnifiedLunchCalendar.tsx — líneas 556-566

```
cancellation_reason: "AUTO: transacción de almuerzo fallida en UnifiedLunchCalendar"
```

C) src/components/teacher/TeacherLunchCalendar.tsx — líneas 319-327

```
cancellation_reason: "AUTO: transacción de almuerzo fallida en TeacherLunchCalendar"
```

Nota: Teacher.tsx ya usa UnifiedLunchCalendarV2 (RPC atómica). TeacherLunchCalendar.tsx sigue en repo pero NO es la ruta activa de profesores.

D) src/components/parent/LunchOrderCalendar.tsx — líneas 668-676

```
// Batch: si falla insert masivo de transactions, cancela TODOS los pedidos del batch
.update({ is_cancelled: true, status: "cancelled" }).in("id", orphanedIds)
```

CRÍTICO: LunchOrderCalendar sigue montado en Index.tsx (modal showMenuModal, línea 1150) y se abre desde StudentCard onViewMenu (línea 956). Convive con UnifiedLunchCalendarV2.

E) src/components/lunch/LunchDeliveryDashboard.tsx — líneas 882-892

```
cancellation_reason: "AUTO: transacción fallida en delivery_no_order"
```

Flujo "Agregar almuerzo sin pedido" en cocina — también usa patrón 2 pasos.

1.3 RPC atómica (correcta) vs rutas legacy

supabase/migrations/20260615_create_lunch_order_v2_atomic_rpc.sql

create_lunch_order_v2 inserta lunch_order + transaction en UNA transacción SQL. Si falla, rollback total — no hay huérfano ni auto-cancel.

src/components/lunch/UnifiedLunchCalendarV2.tsx — líneas 1503-1527

Padres y profesores en flujo principal usan create_lunch_order_v2. NO contiene lógica AUTO-cancel.

PROBLEMA: padres aún pueden entrar al modal LunchOrderCalendar (legacy) de patrón peligroso de batch cancel.

1.4 Por qué falla la transacción en hora punta (7–9 AM)

La migración 20260615 documenta la causa: fn_sync_student_balance hace SUM(amount) sobre transactions pending por student_id. Sin índice cubriente, bajo concurrencia matutina el `adv timeout !` INSERT transactions falla.

- Índice propuesto: `idx_transactions_balance_sync_covering (student_id, payment_status) INCLUDE (amount)`.
- `resilientFetch (src/lib/resilientFetch.ts)` NO reintenta POST/RPC — correcto financieramente, pero amplifica percepción de fallo ante timeout de red.
- `tg_enforce_spending_limit BYPASS` almuerzos si `metadata.lunch_order_id IS NOT NULL (20260428)` — descartado como causa de bloqueo.

1.5 Bug de visualización en cocina (parece "anulación automática")

src/components/lunch/LunchDeliveryDashboard.tsx — líneas 1836-1850

Fetch inicial filtra `is_active_unified=true` (excluye cancelados). Pero Realtime en UPDATE parchea la fila IN SITU con `is_cancelled=true` sin removerla. Resultado: pedido aparece en cocina y luego se muestra como Anulado tras fallo de deuda.

Gestión LunchOrders.tsx Sí puede listar anulados cuando el filtro lo permite (líneas 538-542, 2474-2475).

TASK 2 — Auditoría de actualización y caché

2.1 vercel.json — cabeceras Cache-Control

- `/index.html!` no-cache, no-store, must-revalidate (OK).
- `/version.json!` no-cache + Pragma + Expires:0 (OK).
- `/sw.js!` no-cache (OK).
- `/manifest.webmanifest!` no-cache (OK).

PROBLEMA: regla catch-all `"/*(*)"` solo aplica CSP/HSTS — NO define Cache-Control para assets hashed (`/assets/index-*.js`). Vercel CDN aplica cache immutable por defecto en assets con hash (acceptable). El riesgo real NO es HTML persistente sino JS en memoria + Service Worker precache.

2.2 PWA / Service Worker (`vite.config.ts`)

```
registerType: "autoUpdate"
workbox: { skipWaiting: true, clientsClaim: true }
globPatterns: ["**/*.js,css,html,..."]
runtimeCaching: {
  version.json! NetworkOnly
  *.supabase.co! NetworkOnly
}
```

El SW nuevo SÍ se activa sin estado waiting. PERO el bundle React ya cargado en memoria NO se reemplaza hasta `location.reload()`.

2.3 VersionChecker — por qué el botón no aparece o no soluciona

`src/components/VersionChecker.tsx`

- Líneas 18-22: auto-reload ELIMINADO intencionalmente ("no romper wizard de almuerzos").
- Líneas 92-115: solo muestra toast con botón "Actualizar ahora" — acción manual.
- Líneas 54-89: si `/version.json` falla (`HTTP`200`, `Content-Type HTML` por rewr... notificación.
- Líneas 124-126: primera lectura solo guarda versión baseline; detecta cambio en lecturas posteriores (intervalo 60s, delay inicial 15s).
- Usuarios con PWA instalada en build anterior a `VersionChecker`: no tienen el botón.
- Toast viewport (`toast.tsx L17`): en móvil aparece arriba (top-0); puede quedar oculto tras teclado/navbar.
- Dashboard `handleForceUpdate` (`Dashboard.tsx L576-586`): envía broadcast pero `VersionChecker` solo muestra toast — NO recarga automáticamente (contradice mensaje admin "recargarán automáticamente").

2.4 Inconsistencias detectadas

- `index.html` referencia `/manifest.json`; `VitePWA` genera `manifest.webmanifest` — `vercel.json` cachea `.webmanifest` pero no `.json` duplicado.
- `main.tsx` comentario L62-64 desactualizado (dice `NetworkFirst Supabase`; real es `NetworkOnly`).
- `package.json` version 1.22.0 vs `app.config.ts` version 1.9.1 — divergencia cosmética, no afecta `version.json` (usa `app.config`).

TASK 3 — Solución estructural (Enterprise)

3.1 Backend / datos — mitigar anulaciones

P0 — Verificar despliegue en producción

- Confirmar que `idx_transactions_balance_sync_covering` y `create_lunch_order_v2` están aplicados en Supabase prod (migración 20260615).

P0 — Eliminar rutas legacy de creación 2 pasos

- Retirar `LunchOrderCalendar` modal de `Index.tsx` (línea 1150) o redirigir a `UnifiedLunchCalendarV2`.
- Migrar `LunchDeliveryDashboard AddWithoutOrderModal` a RPC atómica.
- Deprecar `OrderLunchMenus`, `UnifiedLunchCalendar`, `TeacherLunchCalendar` si aún referenciados.

P1 — RPC backend de compensación (no cancelar en frontend)

Si `create_lunch_order_v2` falla por timeout de red pero el pedido SÍ se creó, el cliente no debe hacer UPDATE cancel. Patrón: RPC `get_order_status(order_id)` + idempotencia por `(student, date, category)`.

P1 — Ventana hora crítica (opcional negocio)

Ampliar `global_lunch_deadline_time` en `system_status` solo bloquea nuevos INSERT (trigger BEFORE INSERT). NO evita auto-cancel frontend. La palanca real es performance de `fn_sync_student_balance` + atomicidad.

P1 — Fix Realtime cocina

En `LunchDeliveryDashboard` handler UPDATE: si `is_cancelled=true` OR `status=cancelled` patch in-place.

3.2 Frontend — Zero-Manual-Refresh (configuración exacta)

A) vercel.json — añadir headers para assets

```
{
  "source": "/assets/(.*)",
  "headers": [{ "key": "Cache-Control",
    "value": "public, max-age=31536000, immutable" }]
}
// Mantener index.html, version.json, sw.js en no-cache
```

B) vite.config.ts — activación + recarga controlada

```
VitePWA({ registerType: "prompt" }) // o mantener autoUpdate
// Añadir en main.tsx listener:
navigator.serviceWorker.addEventListener("controllerchange", () => {
  if (sessionStorage.getItem("sw_reload_done")) return;
  sessionStorage.setItem("sw_reload_done", "1");
  window.location.reload();
});
```

Alternativa menos agresiva: recargar solo si `document.visibilityState===visible` y no hay wizard activo (`sessionStorage lunch_wizard_*`).

C) VersionChecker — política enterprise

- Detectar `controllerchange` del SW !' reload automático fuera de flujos críticos
- Mantener toast solo como fallback si `version.json` cambia pero SW no actualizó.
- Reducir `INITIAL_DELAY_MS` de 15s a 0 en producción.

3.3 Consultas forenses recomendadas (producción)

```
SELECT cancellation_reason, COUNT(*)
FROM lunch_orders
WHERE created_at >= (timezone("America/Lima", now()))::date)::timestampz
AND is_cancelled = true
GROUP BY 1 ORDER BY 2 DESC;
-- Esperado: filas AUTO: transacción... en horas 07-09
```

3.4 Riesgos si no se corrige

- Pedidos válidos cancelados en pico !' cocina sin comida registrada !' reclamos
- Builds viejos en celulares !' bugs ya corregidos no llegan !' soporte eterno.
- Datos Supabase frescos + UI vieja !' estados contradictorios (parece anulado